VLM Reliability Toolkit

Presenter script: how to explain the application flow, and what to show in the live demo

Part	Content	Time
A	One-minute pitch (the problem and our idea)	1 min
B	Application flow, step by step, with what to say	3 min
C	Live demo run sheet (exact inputs, where to click, what to point at)	5 min
D	Benchmarks (POPE and CHAIR) talk track	1–2 min
E	Likely questions and short answers	as needed
F	Checklists: tonight, demo morning, and fallbacks	—

How to read this script

- Text in a **purple SAY box** is meant to be spoken. Say it in your own words; do not read it word for word.
- Plain text is stage direction: what to click, what to point at, and why.
- Suggested speakers (change freely): **Prithvi**, pitch and flow; **Kavyansh**, live demo; **Harsh**, re-ranking and human review; **Yash**, benchmarks and questions.

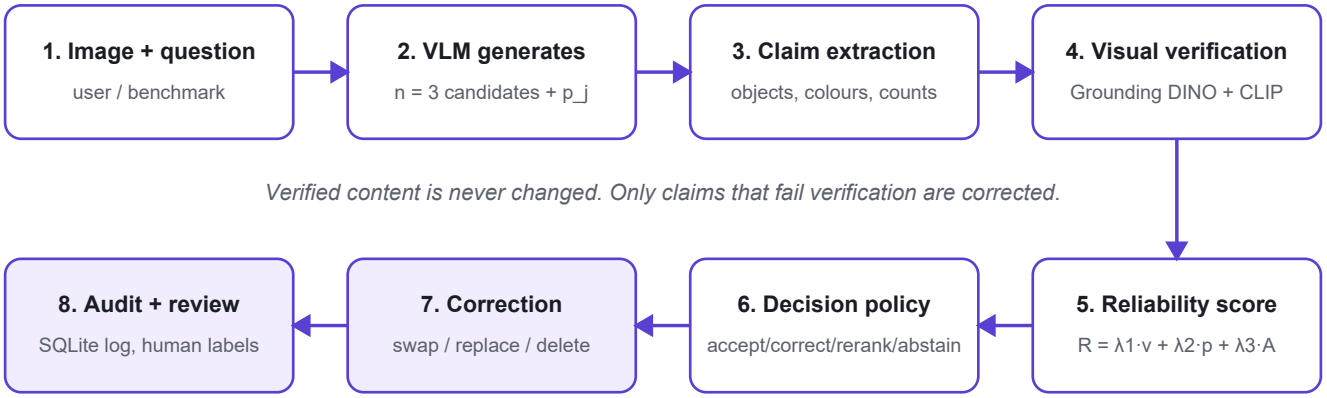
A. One-minute pitch

SAY — Prithvi: Vision-language models such as LLaVA or Gemini describe images fluently, but they make two kinds of mistakes. First, they **hallucinate objects**: they mention a surfboard that is not in the picture. Second, they **mis-bind attributes**: they say *blue shirt and red jeans* when the shirt is red and the jeans are blue. Retraining the model to fix this is expensive and has to be repeated for every model.

SAY: Our toolkit is a **reliability layer that sits after any VLM**. It needs no training and works with any model. It breaks the answer into small checkable claims, checks each claim against the image with separate vision models, scores how reliable each answer is, and then accepts it, corrects it, picks a better candidate, or abstains. Anything it cannot verify is flagged for a human, and everything is logged.

B. Application flow

Draw or show this diagram first, then walk through the eight steps.



1. Input

The user uploads an image and asks a question, for example *Describe this image*. In the benchmarks, the image and question come from the POPE or CHAIR dataset.

> **SAY:** The input is an image plus a question. It can come from a person using our dashboard, or from a benchmark.

In the code: Dashboard (Next.js) → FastAPI /api/analyze

2. Generate n candidates

The VLM writes three answers: one greedy (the normal answer) and two sampled at temperature 0.7. For local models we also record p_j, the model's own confidence: the average token probability.

> **SAY:** We don't trust a single answer. We ask the model for three. The first is exactly what the model would normally say, so it is our baseline. The other two give us alternatives and let us measure agreement.

In the code: vlm.py (Gemini, OpenAI, Groq), models/hf_vlm.py (LLaVA on GPU)

3. Claim extraction

Each answer is split into atomic claims: existence claims (*there is a dog*), binding claims (*the shirt is red*), counts (*two dogs*) and spatial relations.

> **SAY:** A sentence can be partly right and partly wrong, so we check it claim by claim. *A man in a red shirt holding a surfboard* becomes: man exists, shirt exists, shirt is red, surfboard exists.

In the code: claims/extractor.py, using the 80 COCO object classes and their synonyms

4. Visual verification

Existence: Grounding DINO searches the image for the object. Its best box confidence is the score (paper Eq. 1). Binding: we crop that object's box and ask CLIP how likely the colour or material is *inside that crop*, multiplied by the detection confidence (Eq. 2).

> **SAY:** This is the key idea. We check the colour on the object's **own crop**, not on the whole image. That way, a blue sky can't make *blue shirt* look correct. That is how we catch swapped attributes.

In the code: verification/grounding.py, attribute_verifier.py, service.py

5. Reliability score

Each candidate gets $R = \lambda_1 \cdot$(mean claim score) + $\lambda_2 \cdot p_j$ + $\lambda_3 \cdot$(agreement with the other candidates), with weights 0.6 / 0.2 / 0.2 (Eq. 3). If a term is unavailable, for example p_j for an API model, its weight is shared across the other terms.

> **SAY:** Each answer gets one reliability number between 0 and 1. It combines three signals: how well its claims are verified, how confident the model was, and how much the three answers agree with each other.

In the code: reliability/engine.py

6. Decision policy

Per claim: score ≥ τ_{hi} → ACCEPT; score ≤ τ_{lo} → CORRECT; in between → ABSTAIN (flag for review). Per answer: if the top answer's R is below 0.5 and another candidate passes, RERANK to that candidate. The thresholds are tuned on a held-out set, never on the test data.

> **SAY:** Each claim gets one of four actions. Accept means the claim is clearly true. Correct means it is clearly false and we know the fix. Abstain means we are not sure, so we don't guess and flag it for a human. Re-rank means another candidate is clearly more reliable, so we use that one instead.

In the code: core/policy.py, core/pipeline.py

7. Correction

Edits only the failed spans: it swaps mis-bound attributes (red ↔ blue), replaces a wrong colour with the verified one, and deletes hallucinated objects together with their phrase.

> **SAY:** We make the smallest possible edit. If a claim is verified, its words are never touched.

In the code: correction/engine.py

8. Output, audit and human review

The final answer is returned with every claim, score, box and action. Each run is saved with the image hash. Flagged claims appear on the Review page, where a person can label them.

> **SAY:** Everything is traceable. For each answer you can see which claim was checked, the evidence, the score, and what we did. The uncertain cases go to a person, and their labels are stored for later calibration.

In the code: database/db.py (SQLite), Review page in the dashboard

C. Live demo run sheet (about 5 minutes)

Open **http://127.0.0.1:3000/analyze**. Run one analysis before the audience arrives so the models are already loaded. The first run takes about a minute; later runs take a few seconds.

Scene 1 — Paper Table IV: attribute swap and object deletion (90 s)

- Upload **test_images/table_iv.jpg**, a photo of a person in a red shirt and blue jeans.
- Open **Controlled candidate override** and paste:
  A person in a blue shirt and red jeans stands on the beach holding a surfboard.
- Click **Run reliability analysis**.
- Point at the **Evidence canvas**: the Grounding DINO boxes for the person, shirt and jeans, and no box for a surfboard.
- Point at the **claims table**: person ACCEPT; shirt CORRECT (swap); jeans CORRECT (swap); surfboard CORRECT (delete). Show the crop thumbnail next to the shirt claim.
- Point at the **Correction diff**: *A person in a red shirt and blue jeans stands on the beach.*

> **SAY — Kavyansh:** We typed a deliberately wrong caption so you can see exactly what happens. The toolkit found the shirt and jeans, checked the colour inside each crop, saw the colours were swapped and swapped them back. It found no surfboard, so it deleted that phrase. Everything else in the sentence stays exactly as it was. This is the example from Table IV in our paper.

Scene 2 — Live model: a real hallucination (60 s, optional)

- Clear the override. Upload **street.jpg** and ask *Describe this image in detail.* Gemini generates the three candidates live.
- Show any claim that was deleted or flagged. If Gemini happens to be fully correct, say so. That is also a valid result: verified answers pass through unchanged.
- If the network is slow, skip this scene; the override demos are enough.

Scene 3 — Re-ranking (60 s)

- Upload **coco_snow.jpg** (or any snow image). In the override, paste two candidates separated by **|||**:
  A skier in a red jacket holds a snowboard next to a dog. ||| *A person with a snowboard stands on the snow.*
- Open the **Candidates** page: candidate 1 has a low R with failed claims; candidate 2 is selected with decision **RERANK** and the reason is shown.
- Show the reliability panel: mean v, agreement A, and p (*n/a (API model)* for overrides and Gemini).

> **SAY — Harsh:** The first answer invents a dog and a red jacket. Its reliability score is below 0.5. The second answer is simpler but fully verified, so the toolkit selects it instead of patching the first one. That is the re-rank rule from the paper.

Scene 4 — Abstention and human review (45 s)

- Upload **small_object.jpg** with a caption that mentions the small or hidden object.
- Open **Review**: the flagged claim appears with its crop, score and explanation.
- Click **Claim is true** (or false), reload the page, and show that the label persisted.

> **SAY — Harsh:** When the evidence is in the uncertain zone, we don't guess. The claim is flagged, a person makes the call, and that label is stored in the audit log for later calibration.

Scene 5 — Benchmarks (60–90 s)

- Open the **Evaluation** page and use the talk track in part D.

D. Benchmarks talk track

> **SAY — Yash:** To measure this at scale, we ran LLaVA-1.5-7B on two standard hallucination benchmarks on free Kaggle GPUs. **POPE** asks yes-or-no questions such as *Is there a dog in the image?* over 450 COCO images, in three difficulty splits: random, popular and adversarial. **CHAIR** asks the model to caption 500 images and counts how many mentioned objects are not really there.

Metric	What it means	Better
POPE accuracy / F1	Correct yes/no answers about object presence	Higher
POPE yes-ratio	How often the model says yes. Hallucinating models say yes too often.	Closer to 50%
$CHAIR_S$	Share of captions with at least one hallucinated object	Lower
$CHAIR_I$	Share of mentioned objects that are hallucinated	Lower
Object recall	Real objects still mentioned. Checks we didn't over-delete.	Higher
Latency ratio	Toolkit time ÷ base model time	Lower

> **SAY:** Compare the **base VLM** row with the **full toolkit** row. The ablation rows switch off one part at a time (attribute binding, the reliability score, or multiple candidates) to show what each part contributes. Thresholds were tuned on 50 held-out images that are never in the test set. One honest note: POPE and CHAIR only measure **object** hallucination. The attribute-binding correction is what you saw in the live demo.

Read the actual numbers off results/RESULTS.md after the Kaggle run. Don't quote numbers from memory. If the run is only partly finished, say it is a partial run and give the sample count.

E. Likely questions

Q: Why not just fine-tune the model?

Fine-tuning is costly and has to be redone for every model and version. Our layer is training-free and model-agnostic. We use the same code for Gemini, OpenAI and LLaVA.

Q: Isn't the verifier also a model that can be wrong?

Yes. That's why we don't trust any single score. We use per-type thresholds tuned on held-out data, abstain in the uncertain band, and send those cases to a human. Grounding DINO and CLIP are trained for detection and matching, which is a narrower and more reliable task than open-ended generation.

Q: Why three candidates?

Agreement between independent samples is a strong signal of reliability, and a second candidate gives us something to re-rank to. The *n = 1* ablation shows what is lost without it.

Q: Why crop before checking colour?

If you ask CLIP about the whole image, any blue in the scene can support *blue shirt*. Checking inside the object's box ties the attribute to that object. That is the binding problem.

Q: How much slower is it?

We report the latency ratio in the benchmark table. Verification results are cached per image, so repeated analysis and ablations add almost no extra cost.

Q: What can't it do?

It doesn't fix reasoning errors, small or hidden objects the detector misses, or out-of-vocabulary objects. Claim-extraction errors carry through, and colour depends on lighting. These are listed as limitations in the paper.

Q: Is p_j available for Gemini?

No. Gemini doesn't return token probabilities, so that term is marked n/a and its weight is shared across verification and agreement. For LLaVA in the benchmarks, p_j is computed from the real token probabilities.

Q: Where do the thresholds come from?

From evaluation/tune.py on a separate 50-image held-out POPE split: τ_{lo} maximises F1, τ_{hi} targets 95% precision for accepted claims, and the λ weights minimise calibration error. They are saved to results/tuned.yaml.

F. Checklists

Kaggle run

Job (runs in parallel)	Work	Estimate
Setup (cells 1–3)	Install packages, ~1,900 COCO images, ~16 GB of model weights	20–30 min
POPE on GPU 0	~8,400 yes/no answers (8 tokens each), verification, tuning, 5 ablations	1.5–2.5 h
CHAIR on GPU 1	500 images × 3 captions (128 tokens each), verification, 5 ablations	2.5–3.5 h
Total	Bounded by CHAIR. Kaggle's limit is 12 h per run.	≈ 3.5–4.5 h

Check progress about 30 minutes in. results/tuned.yaml should appear in the Hugging Face repo Prithxvhie44/vlm-results. If the run stops, commit again and it resumes.

Tonight

- Start the full Kaggle run: BENCHES = ["pope", "chair"], then **Save Version → Save & Run All (Commit)**.
- Put the four images in test_images/: table_iv.jpg, street.jpg, small_object.jpg, two_dogs.jpg (plus coco_snow.jpg).
- Rehearse scenes 1–4 once and check that each image gives the expected result. If one doesn't, change the photo, not the thresholds.

Demo morning (30 minutes before)

- hf download Prithxvhie44/vlm-results --repo-type dataset --local-dir . --include "results/*"
- python -m evaluation.run_benchmarks --proc results/proc --out results/benchmark_report.json --markdown results/RESULTS.md
- powershell -ExecutionPolicy Bypass -File scripts\dev.ps1, then run one warm-up analysis.
- Check that the Evaluation page shows the tables. Keep RESULTS.md open in a second tab.
- Set up Kavyansh's MacBook the same way as a backup machine.

If something breaks

Problem	What to do
No Wi-Fi or the Gemini key fails	Use the candidate override. It needs no API key, only the cached models.
Dashboard won't start	Run python demo_paper_example.py. It prints the Table IV trace with no models and no network.
Kaggle run not finished	Show the partial results on Hugging Face and state the sample count.
Laptop fails	Switch to the MacBook backup.